COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 16, 2026	**DATE SUBMITTED:**	July 21, 2026
NAME:	Jaime D.C. Godino Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

CODE

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
  Student(std::string newName ="John Doe", int newAge=18){
    studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
    };
    //deconstructor
    ~Student(){
      std::cout << "Destructor Called." << std::endl;
    }

    //copy constructor
```

```cpp
    Student(const Student &copyStudent){
      std::cout << "Copy Constructor Called" << std::endl;
      studentName = copyStudent.studentName;
      studentAge = copyStudent.studentAge;
    }

    //Display Attributes
    void printDetails(){
      std::cout << this->studentName << " " << this->studentAge << std::endl;
    }

};

int main() {
  Student student1("Roman", 28);
  Student student2(student1);
  Student student3;
  student3 = student2;

  return 0;
}
```

OUTPUT

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

I observed that the output shows that when student1 is created, the constructor is called, when student 2 is initialized using student1, the copy constructor is invoked to duplicate the object's data. The object student 3 is created using the default constructor, while student3 = student2 does not produce any output because it uses the compiler-generated copy assignment operator rather than the copy constructor. Since printDetails() is never called, no student information is ever displayed. At the end of the program, the destructor is called three times in reverse order of object creation.

Table 2-1. Initial Driver Program

CODE

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
   Student(std::string newName ="John Doe", int newAge=18){
    studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
    };
    //deconstructor
    ~Student(){
      std::cout << "Destructor Called." << std::endl;
    }

    //copy constructor
    Student(const Student &copyStudent){
      std::cout << "Copy Constructor Called" << std::endl;
      studentName = copyStudent.studentName;
      studentAge = copyStudent.studentAge;
    }

    //Display Attributes
    void printDetails(){
      std::cout << this->studentName << " " << this->studentAge << std::endl;
    }

};

int main() {
  const size_t j = 5;

  Student studentList[j] = {};
  std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
  int ageList[j] = {15, 16, 18, 19, 16};

  return 0;
```

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

In the output, it shows that the default constructor is called five times because the array studentList contains five Student objects, causing each object to use the default constructor with “John Doe” and 18. Although, the nameList and ageList arrays are created, they are not used to initialize Student objects so they do not affect the output whatsoever. Just like the previous code, the printDetails() is never called and no student information is displayed. When the program is terminated, the destructor is automatically called five times in reverse order of object creation.

Table 2-2. Modified Driver Program with Student Lists

LOOP A

```
42      for(int i = 0; i < j; i++){ //loop A
43          Student *ptr = new Student(namesList[i], ageList[i]);
44          studentList[i] = *ptr;
45      }
```

OBSERVATION

Loop A dynamically creates a new Student object during each iteration using the names and ages stored in namesList and ageList, causing the constructor to be called five times. Each created object is then assigned to the corresponding element studentList using the copy assignment operator, not the copy constructor, since the objects in studentList were already constructed earlier. Because the dynamically allocated objects are never deleted with delete, their destructors are not called during the loop.

LOOP B

```
47    for(int i = 0; i < j; i++){ //loop B
48        studentList[i].printDetails();
49    }
```

OBSERVATION

Loop B, on the other hand, iterates through the studentList array and calls the printDetails() function for each Student object. Since the objects were assigned the data from the dynamically created students in Loop A, the program displays the updated names and ages instead of the default values, This loop only accesses and prints existing objects without creating or destroying new objects.

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

The output shows that the constructor is called five times when the studentList array is created with default

values and five more times in Loop A when new Student objects are dynamically allocated using the constructor. Loop B successfully prints the assigned student names and ages, confirming that the values from the dynamically created objects were copied into the corresponding elements of studentList. Finally, only five destructor calls are displayed because only the objects in studentList are automatically destroyed at the end of the program, while dynamically allocated objects created with new are never deleted.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code

```cpp
#include <iostream>
#include <string.h>

class Grocery{
protected:
    std::string name;
    double price;
    int quantity;

public:
    //constructor
    Grocery(std::string n = "", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    //copy constructor
    Grocery(const Grocery& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    //copy assignment operator
    Grocery& operator=(const Grocery& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }
    //destructor
    virtual ~Grocery() {
        std::cout<< name <<" destroyed" <<std::endl;
    }

```

```cpp
    double calculateSum() const {
        return price * quantity;
    }

    virtual void display() const {
        std::cout << "Name: " << name << std::endl;
        std::cout << "Price: PHP " << price << std::endl;
        std::cout << "Quantity: " << quantity << std::endl;
        std::cout << "Total: PHP " << calculateSum() << std::endl << std::endl;
    }
};

class Fruit : public Grocery {
public:
    Fruit(std::string n = "", double p = 0, int q = 0)
        : Grocery(n, p, q) {}

    Fruit& operator=(const Fruit& other) {
        Grocery::operator=(other);
        return *this;
    }

    ~Fruit() {
        std::cout << "Fruit Destructor Called" << std::endl << std::endl;
    }

    void display() const override {
        std::cout << "Fruit" << std::endl;
        Grocery::display();
    }
};

class Vegetable : public Grocery {
public:
    Vegetable(std::string n = "", double p = 0, int q = 0)
        : Grocery(n, p, q) {}

```

```
    Vegetable(const Vegetable& other)
        : Grocery(other) {}

    Vegetable& operator=(const Vegetable& other) {
        Grocery::operator=(other);
        return *this;
    }

    ~Vegetable() {
        std::cout << "Vegetable Destructor Called" << std::endl << std::endl;
    }

    void display() const override {
        std::cout << "Vegetable" << std::endl;
        Grocery::display();
    }
};

double TotalSum(Grocery* GroceryList[], int size) {
    double total = 0;

    for (int i = 0; i < size; i++) {
        if (GroceryList[i] != nullptr)
            total += GroceryList[i]->calculateSum();
    }

    return total;
}

int main() {

    Grocery* GroceryList[4];

    GroceryList[0] = new Fruit("Apple", 10, 7);
    GroceryList[1] = new Fruit("Banana", 10, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50, 10);
```

```
113
114     std::cout << "Jenna's Grocery List\n";
115
116     for (int i = 0; i < 4; i++) {
117         GroceryList[i]->display();
118     }
119
120     std::cout << "Total Amount to Pay: PHP "
121               << TotalSum(GroceryList, 4) << std::endl;
122
123     std::cout << "\nDeleting Lettuce\n\n";
124
125     delete GroceryList[3];
126     GroceryList[3] = nullptr;
127
128     for (int i = 0; i < 4; i++) {
129         if (GroceryList[i] != nullptr)
130             GroceryList[i]->display();
131     }
132
133     std::cout << "Updated Total: PHP " << TotalSum(GroceryList, 4) << std::endl << std::endl;
134
135     //Delete
136     for (int i = 0; i <  4; i++) {
137         delete GroceryList[i];
138     }
139     return 0;
140 }
```

Output

```
Jenna's Grocery List
Fruit
Name: Apple
Price: PHP 10
Quantity: 7
Total: PHP 70

Fruit
Name: Banana
Price: PHP 10
Quantity: 8
Total: PHP 80

Vegetable
Name: Broccoli
Price: PHP 60
Quantity: 12
Total: PHP 720

Vegetable
Name: Lettuce
Price: PHP 50
Quantity: 10
Total: PHP 500

Total Amount to Pay: PHP 1370

Deleting Lettuce
```

```
Vegetable Destructor Called

Lettuce destroyed
Fruit
Name: Apple
Price: PHP 10
Quantity: 7
Total: PHP 70

Fruit
Name: Banana
Price: PHP 10
Quantity: 8
Total: PHP 80

Vegetable
Name: Broccoli
Price: PHP 60
Quantity: 12
Total: PHP 720

Updated Total: PHP 870
```

```
Fruit Destructor Called

Apple destroyed
Fruit Destructor Called

Banana destroyed
Vegetable Destructor Called

Broccoli destroyed
```

Conclusion:
In this activity, I learned how to create classes with constructors, copy constructors, copy assignment operators, and destructors. I also experienced using dynamic memory allocation with new and delete. Additionally, I learned the importance of properly deallocating memory to prevent memory leaks and using nullptr to avoid pointing to a deleted object.

The code begins with creating the Grocery parent class, then creating the child classes named Fruit and Vegetable. After defining the classes, the program dynamically allocated objects and stored them in the GroceryList array, and computed the total cost using the TotalSum() function. Finally, it deleted the Lettuce object and recalculated the total.

I was able to implement the required classes, calculate the total amount, and correctly remove an item from the grocery list in this activity. While I was coding, I encountered an issue where I could not update the total after deleting the lettuce; eventually, I did find a way to solve the problem. There are still a lot of areas for improvement, and there are ways to improve, such as to keep practicing and coding each day.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Formatting, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Using ChatGPT, I asked how to format a parent class and how a child class can inherit it in C++. Then, I used Grammarly to fix my grammar when it comes to the code analysis and conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.